UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI
DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY

MASTER THESIS

By

Dao Xuan Quy - 2440053

Title:

ROAD SEGMENTATION FOR AUTOCAR NETWORKS

Supervisor: Dr. Nghiem Thi Phuong

Hanoi, August – 2026

To whom it may concern,

I, Nghiem Thi Phuong, certify that the internship report of Mr. Dao Xuan Quy is qualified to be presented in the Internship Jury 2025-2026.

Hanoi, August 2026

Supervisor's signature

Nghiem Thi Phuong

Contents

List of Figures

List of Tables

Declaration

I hereby, Dao Xuan Quy, declare that my thesis represents my own work after doing an internship at USTH ICTLAB, University of Science and Technology of Hanoi.

I have read the University's internship guidelines. In the case of plagiarism appearing in my thesis, I accept responsibility for the conduct of the procedures in accordance with the University's Committee.

Hanoi, August 2026

Signature

Dao Xuan Quy

Acknowledgements

I would like to express my sincere gratitude to Dr. Nghiem Thi Phuong and A.Prof. Tran Giang Son for they exceptional guidance and support throughout this project. I am also grateful to USTH for providing me with the computational resources needed to complete this research, and to the open-source communities behind the KAN, BiKA and U-KAN projects for providing foundational implementations.

Abstract

Semantic segmentation is a critical component of autonomous driving perception, requiring both high accuracy and low latency for real-time deployment. Traditional segmentation networks based on U-Net [2], and Transformer architectures rely on Multi-Layer Perceptrons (MLPs) with fixed activation functions, limiting their representational flexibility. On the other hand, lightweight models often suffer from accuracy loss when handling more complicated classes. In this thesis, we study and implement two novel approaches that integrate Kolmogorov-Arnold Networks (KANs) into segmentation architectures for road segmentation. The main contributions of this thesis include:

- Optimizing and Evaluating U-KAN (UKAN), a U-Net style segmentation network that replaces standard MLP layers with KAN layers using learnable activation functions, including multiple KAN variants: FasterKAN (RSWAf basis) , ReLU-KAN, HardSwish-KAN, PWLO-KAN, and TeLU-KAN.
- Implementing and evaluating BiKA (Binarized KAN), a novel architecture that combines the KAN philosophy of per-connection learnable activations with Binary Neural Network (BNN) techniques, achieving extreme compression through 1-bit binarized weights and activations.
- Improving BiKA [3] GPU performance by apply GPU optimization methods to the official implementation
- Benchmarking both architectures against standard baselines in terms of segmentation accuracy (mIoU, Dice), model size, and inference throughput with two distinct problems: Segment Everything (20 classes) and Segment the drivable area only (2 classes)

The experimental results demonstrate that KAN-based architectures successfully achieve a highly accurate segmentation results for multi-class segmentation with UKAN architectures, and retain acceptable accuracy while perform extremely fast with BiKA implementations.

Keywords: Kolmogorov-Arnold Networks, Semantic Segmentation, Binary Neural Networks, Road Segmentation, U-Net, Edge Deployment.

Chapter 1

Introduction

1.1 Context and motivation

1.1.1 Context

Semantic segmentation, the task of assigning a class label to every pixel in an image, is a foundational capability for autonomous driving perception systems. Accurate road segmentation enables autonomous vehicles to identify drivable surfaces, lane boundaries, sidewalks, and other critical scene elements in real time. The evolution of segmentation architectures has progressed from early fully convolutional networks (FCN) [4] to the U-Net encoder-decoder architecture [2], and more recently to Transformer-based models such as SegFormer [5], Swin-UNet [6] and Segment Anything [7].

However, all of these architectures share a fundamental design choice inherited from Multi-Layer Perceptrons (MLPs): they place fixed, pre-defined activation functions (e.g., ReLU, GELU) at the *nodes* of the network, while the *edges* (connections) carry only learnable linear weights,. The Kolmogorov-Arnold Representation Theorem [8, 9] suggests an alternative: replace every linear weight with a non-linear activation functions on the *edges* themselves, allowing the network to learn both the weights and the optimal activation shape for each connection, which as a results, achieve better accuracy and interpretability. This principle forms the foundation of Kolmogorov-Arnold Networks (KANs) [1].

In the Sematic Segmentation contenxt, recent work has demonstrated the potential of KAN-based architectures in medical image segmentation [10]. On the other hand, KAN application to road scene segmentation and their compatibility with extreme model compression techniques such as Binary Neural Networks (BNNs) [11, 12] remain largely unexplored.

1.1.2 Problem Statement

Deploying segmentation models on autonomous vehicles poses two competing requirements. First, the model must achieve high segmentation accuracy to to ensures safeness to vehicles and the passengers. Second, since the car **sees** the environment throught a camera, the model itself have to be capable of processing a large amount of frames per second; therefore, require low latency, small model size, and minimal memory bandwidth.

Standard full-precision segmentation networks (32-bit floating point) achieve high accuracy but require significant computational resources. Binary Neural Networks (BNNs) offer extreme compression by quantizing both weights and activations to 1-bit representations, but they suffer from severe accuracy degradation due to the limited expressiveness of the sign function used for

binarization. The central challenge addressed in this thesis is: *Can the KAN solve the accuracy loss in efficient, binarized networks, while preserving their computational efficiency advantages?*

1.1.3 Motivation

This research is motivated by the need to bridge the gap between expressive, high-accuracy segmentation models and the efficient model real-time performance. We perform two approaches: UKAN for full-precision KAN-augmented segmentation and BiKA for binarized KAN convolutions—we aim to observe how KAN integration affects the accuracy-efficiency for road segmentation.

1.2 Related Work

The development of semantic segmentation architectures has grown significant for the last decade. In 2015, Long et al. [4] introduced Fully Convolutional Networks (FCN), demonstrating that classification CNNs could be adapted for dense prediction. Ronneberger et al. [2] proposed U-Net, whose symmetric encoder-decoder architecture with skip connections became the dominant standard for biomedical and subsequently autonomous driving segmentation.

Modern segmentation architectures have increasingly adopt attention mechanisms. SegFormer [5] introduced a hierarchical Transformer encoder with efficient self-attention, while Swin-UNet [6] adapted the shifted-window Transformer for U-shaped segmentation. DeepLabV3+ [13] combined atrous spatial pyramid pooling with encoder-decoder structures to capture multi-scale context.

In parallel, the Kolmogorov-Arnold Network (KAN) was proposed by Liu et al. [1] as an alternative to MLPs, placing learnable activation functions on edges rather than nodes. Li et al. [10] subsequently demonstrated U-KAN, integrating KAN layers into the U-Net bottleneck for medical image segmentation. FasterKAN [14] addressed the computational overhead of the original B-spline KAN implementation by replacing B-splines with Reflectional Switch Activation Functions (RSWAf).

On the model compression side, Binary Neural Networks (BNNs) were first formalized by Hubara et al. [15], who demonstrated the feasibility of training deep networks with weights and activations constrained to $+1$ and -1. Subsequent works like XNOR-Net [16] and Bi-Real Net [11] advanced BNNs by introducing real-valued skip connections to stabilize binary training. ReActNet [12] further improved BNN accuracy through learnable activation shifts (RPReLU), while IR-Net [17] proposed Libra-PB for balanced weight binarization, and AdaBin [18] introduced learnable per-channel output scaling. Extending these principles to dense prediction tasks, Group-Net [19] demonstrated that structured binary networks, which dvide operations into multiple homogeneous binary branches, can successfully maintain the rich contextual information required for accurate semantic segmentation.

Our work builds upon these foundations by: (1) adapting U-KAN for road segmentation with multiple efficient KAN variants, and (2) proposing BiKA, which combines the KAN per-connection activation idea with state-of-the-art BNN techniques for extreme efficiency.

1.3 Thesis Objective

The primary objective of this thesis is to implement, evaluate, and compare two KAN-based segmentation architectures for road segmentation problem. Specifically, we aim to:

- Adapt the U-KAN architecture for road segmentation on the BDD100K dataset, exploring multiple KAN linear variants (FasterKAN, ReLU-KAN, HardSwish-KAN, PWLO-KAN, TeLU-KAN) to assess their impact on segmentation accuracy and inference speed.
- Design and implement BiKASegNet, a binarized segmentation network that applies the KAN algorithm through per-connection binary thresholds to achieve extreme model compression while retaining acceptable accuracy.
- Benchmark both architectures against standard baseline models in terms of segmentation quality (mIoU, Dice), model size, and inference speed on CUDA platforms.

1.4 Thesis Structure

The remainder of this thesis is structured as follows:

- **Chapter 1** introduces the context, problem statement, related work, and objectives of this research.
- **Chapter 2** discusses the technical background of Kolmogorov-Arnold Networks, the U-Net segmentation, Binary Neural Networks, and the KAN variant implementations.
- **Chapter 3** explains the methodology, including the UKAN architecture, KAN layer integration, the BiKA binarized segmentation network design.
- **Chapter 4** describes the experimental setup, dataset preparation, training procedures, and performance evaluation of both architectures.
- **Chapter 5** summarizes the content of this thesis and proposes future developments of research.

Chapter 2

Background

This chapter introduces the foundational knowledge that are used to implement KAN-based segmentation networks. These include Kolmogorov-Arnold Networks (KANs), Binary Neural Networks (BNNs), and Binarized Kolmogorov-Arnold Networks (BiKA) concept which sucessfully combine the two architectures.

Binary Neural Networks (BNNs) address the hardware constraints of deep learning by pushing model quantization to its absolute theoretical limit. In a BNN, both the network weights and the intermediate feature activations are constrained to 1-bit representations, typically $\{-1, +1\}$, utilizing the sign function. This severe quantization completely eliminates the need for expensive floating-point multiply-accumulate (MAC) operations. Instead, convolutions and dense layers can be executed using highly efficient bitwise XNOR and popcount operations, drastically reducing memory bandwidth and latency on edge devices like FPGAs and embedded GPUs. Despite these immense computational advantages, the extreme information loss induced by binarizing continuous variables natively limits the expressiveness of BNNs. They often suffer from accuracy compared to full-precision networks and require specialized training methodologies, such as the Straight-Through Estimator (STE) and dense skip connections, to maintain stable gradient flow during backpropagation.

Bridging the gap between the high representational capacity of KANs and the extreme hardware efficiency of BNNs, the Binarized Kolmogorov-Arnold Network (BiKA) introduces a novel paradigm for edge-deployed neural networks. While traditional KANs employ computationally expensive continuous splines and standard BNNs rely on uniform binarization, BiKA replaces the continuous edge functions of KANs with learnable, binary step functions parameterized by individualized connection thresholds. By assigning a unique, learnable bias threshold to every single synapse before binarization, BiKA faithfully implements the KAN philosophy of per-connection activations without requiring floating-point arithmetic during inference. Compared to KANs, BiKA sacrifices the smooth continuity of spline functions to achieve a completely multiply-free computational graph. Compared to traditional BNNs, BiKA achieves significantly higher expressiveness without adding computational complexity by shifting the learning capacity from continuous weight magnitudes to highly localized spatial thresholds. This synthesis provides a highly effective balance, delivering KAN-inspired accuracy within the strict power and resource budgets of binary architectures.

2.1 Kolmogorov-Arnold Networks

Kolmogorov-Arnold Networks (KANs) [1] have been proposed as a novel approach to improve traditional Multi-Layer Perceptrons (MLPs) accuracy, inspired by the Kolmogorov-Arnold repre-

sentation theorem. In a standard MLP, the network learns fixed scalar weights on its connections (edges) and applies pre-defined, fixed non-linear activation functions (such as ReLU or sigmoid) at its neurons (nodes). KANs invert this principle by placing learnable, non-linear activation functions directly on the network edges, while the nodes simply sum the incoming signals. The Kolmogorov-Arnold Representation Theorem [8, 9] states that any multivariate continuous function $f : [0, 1]^n \to \mathbb{R}$ can be represented as a finite composition of continuous functions of a single variable and the binary operation of addition:

$$f(x_1, \ldots, x_n) = \sum_{q=0}^{2n} \Phi_q \left(\sum_{p=1}^{n} \phi_{q,p}(x_p) \right) \tag{2.1}$$

where $\phi_{q,p} : [0, 1] \to \mathbb{R}$ are univariate inner functions and $\Phi_q : \mathbb{R} \to \mathbb{R}$ are outer functions. This theorem guarantees the existence of such a representation but does not prescribe how to learn it.

2.1.1 KAN vs MLP Architecture

Liu et al. [1] proposed Kolmogorov-Arnold Networks as a practical neural network architecture inspired by this theorem. The key different between KANs and traditional MLPs is illustrated in Figure 2.1.

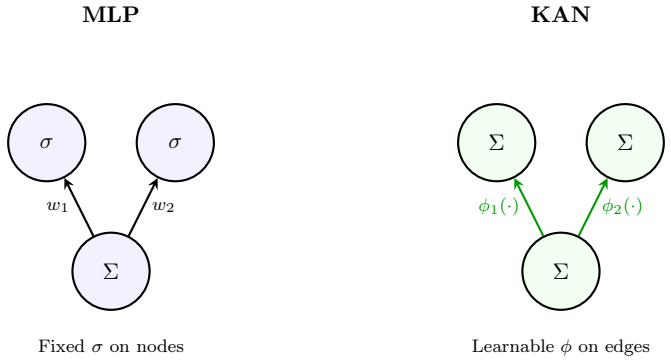


Figure 2.1: Comparison of MLP (fixed activations on nodes, learnable weights on edges) vs KAN (learnable activation functions on edges, summation on nodes) [1].

In an MLP, each neuron computes:

$$y_j = \sigma \left(\sum_i w_{ij} x_i + b_j \right) \tag{2.2}$$

where σ is a fixed activation function (e.g., ReLU) and w_{ij} are learnable scalar weights. In a KAN, each node simply sums its incoming signals, and each edge applies a learnable univariate function:

$$y_j = \sum_i \phi_{ij}(x_i) \tag{2.3}$$

where $\phi_{ij} : \mathbb{R} \to \mathbb{R}$ are learnable activation functions parameterized as spline functions. This places the representational power on the edges rather than the nodes.

2.1.2 Original KAN Implementation with B-Splines

In the original KAN [1], each edge activation ϕ_{ij} is parameterized as a B-spline:

$$\phi_{ij}(x) = w_b \cdot \text{SiLU}(x) + w_s \cdot \text{spline}(x) \tag{2.4}$$

where $\text{SiLU}(x) = x \cdot \sigma(x)$ is a base activation, and $\text{spline}(x) = \sum_k c_k B_k(x)$ is a linear combination of B-spline basis functions B_k with learnable coefficients c_k. The B-spline is defined over a grid of knot points $\{t_0, t_1, \ldots, t_G\}$ spanning the input domain.

While mathematically elegant, B-spline evaluation requires iterative recursion over the grid, resulting in $\mathcal{O}(G \cdot k)$ operations per activation (where G is the grid size and k is the spline order), making the original KAN impractical for high-throughput vision tasks.

2.2 Efficient KAN Variants

To overcome the computational bottleneck of B-splines, several efficient KAN linear replacements have been proposed. Each variant maintains the KAN philosophy of learning activation functions on edges but uses faster basis functions.

2.2.1 FasterKAN (RSWAf Basis)

FasterKAN [14] replaces B-splines with the Reflectional Switch Activation Function (RSWAf). Given a set of grid points $\{g_0, g_1, \ldots, g_{G-1}\}$ uniformly spanning $[g_{\min}, g_{\max}]$ and an inverse denominator parameter β, the basis function is:

$$r_k(x) = 1 - \tanh^2\left((x - g_k) \cdot \beta\right) \tag{2.5}$$

Each basis function r_k produces a smooth, localized bump centred at grid point g_k. The output of a FasterKAN linear layer with input dimension d_{in} and output dimension d_{out} is:

$$y = \mathbf{W}_s \cdot \text{vec}\left(\{r_k(x_i)\}_{k=0,\ldots,G-1}^{i=1,\ldots,d_{\text{in}}}\right) \tag{2.6}$$

where $\mathbf{W}_s \in \mathbb{R}^{d_{\text{out}} \times (d_{\text{in}} \cdot G)}$ is a learnable spline projection matrix, and $\text{vec}(\cdot)$ concatenates all basis evaluations into a single vector. The input is first normalised via LayerNorm. A custom CUDA kernel computes the forward and backward passes of the RSWAf function, achieving significant speedups over the B-spline implementation.

2.2.2 ReLU-KAN

ReLU-KAN [20] replaces the smooth basis with a piecewise-linear function using ReLU:

$$\psi_k(x) = \text{ReLU}(x - g_k) = \max(0, x - g_k) \tag{2.7}$$

This produces a basis of shifted ramp functions. The output is computed as:

$$y = \mathbf{W} \cdot \text{vec}\left(\{\psi_k(x_i)\}_{k,i}\right) \tag{2.8}$$

ReLU-KAN achieves maximal inference speed due to the trivial cost of ReLU evaluation, but trades off smoothness for speed.

2.2.3 HardSwish-KAN

HardSwish-KAN adapts the HardSwish activation [21] as its basis function:

$$\psi_k(x) = \text{HardSwish}(x - g_k) = (x - g_k) \cdot \frac{\text{ReLU6}(x - g_k + 3)}{6} \tag{2.9}$$

This provides a smooth, differentiable basis that avoids the dying neuron problem of ReLU-KAN while maintaining computational efficiency since HardSwish requires no transcendental functions.

2.2.4 TeLU-KAN

TeLU-KAN employs the Hyperbolic Tangent Exponential Linear Unit (TeLU) activation [22]:

$$\psi_k(x) = (x - g_k) \cdot \tanh(\exp(x - g_k)) \tag{2.10}$$

The TeLU activation provides stable gradients across a wide input range, offering a compromise between the smoothness of FasterKAN and the simplicity of ReLU-KAN.

2.2.5 PWLO-KAN (Piecewise Linear Optimisation)

PWLO-KAN draws upon piecewise linear network representations [23] and implements a lookup-table-based approach where each segment of the input domain is associated with a learned affine transformation:

$$\phi_k(x) = a_k \cdot x + b_k, \quad x \in [g_k, g_{k+1}) \tag{2.11}$$

where a_k, b_k are learnable parameters stored as embedding tables. The segment index is computed as:

$$k = \text{clamp}\left(\left\lfloor \frac{x - g_{\min}}{g_{\max} - g_{\min}} \cdot G \right\rfloor, 0, G - 1\right) \tag{2.12}$$

PWLO-KAN is particularly suited for integer-arithmetic inference engines since the lookup and multiply-accumulate operations can be implemented with shift-and-add primitives.

2.3 U-Net Segmentation Architecture

U-Net [2] established the symmetric encoder-decoder architecture with skip connections as the standard for dense prediction tasks. The encoder progressively downsamples the input through convolutional blocks and pooling operations, extracting increasingly abstract feature representations. The decoder upsamples these features back to the original resolution, with skip connections from corresponding encoder stages providing fine-grained spatial detail.

Each encoder stage consists of a double convolution block:

$$\text{ConvLayer}(x) = \text{ReLU}(\text{BN}(\text{Conv}_{3\times3}(\text{ReLU}(\text{BN}(\text{Conv}_{3\times3}(x)))))) \tag{2.13}$$

followed by 2×2 max pooling for spatial downsampling. The decoder mirrors this structure using bilinear upsampling and concatenation (or addition) with encoder skip connections, as illustrated in Figure 2.2.

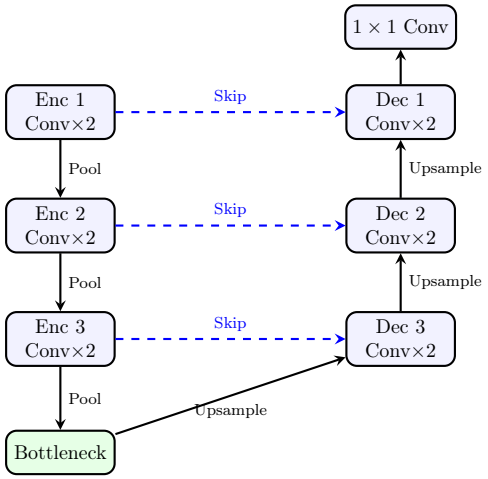


Figure 2.2: Standard U-Net encoder-decoder architecture with skip connections [2].

2.4 Binary Neural Networks

Binary Neural Networks (BNNs) represent the extreme end of model quantization, constraining both weights and activations to $\{+1, -1\}$. The binarization function for weights is:

$$w_b = \text{sign}(w) = \begin{cases} +1 & \text{if } w \geq 0 \\ -1 & \text{otherwise} \end{cases} \tag{2.14}$$

and similarly for activations: $x_b = \text{sign}(x)$. This allows the multiply-accumulate operations in convolutions to be replaced by XNOR and popcount operations:

$$w_b \cdot x_b = \text{XNOR}(w_b, x_b), \quad \sum_i w_{b,i} \cdot x_{b,i} = \text{popcount}(\text{XNOR}(\mathbf{w}_b, \mathbf{x}_b)) \tag{2.15}$$

achieving up to $\sim$58$\times$ theoretical speedup and $\sim$32$\times$ memory reduction compared to full-precision networks [16].

2.4.1 Straight-Through Estimator (STE)

The sign function has zero gradient almost everywhere, making direct backpropagation impossible. Bengio et al. [24] proposed the Straight-Through Estimator (STE), which approximates the gradient of the sign function as:

$$\frac{\partial \mathcal{L}}{\partial w} \approx \frac{\partial \mathcal{L}}{\partial w_b} \cdot \mathbf{1}_{|w| \leq 1} \tag{2.16}$$

where the gradient is passed through unchanged within the clipping window $[-1, 1]$ and zeroed outside. This enables training of binary networks with standard gradient-based optimisers.

2.4.2 Key BNN Techniques

Several techniques have been developed to improve the accuracy of BNNs:

Bi-Real Net Skip Connections

Bi-Real Net [11] introduces full-precision (FP32) identity shortcut connections that bypass the binary convolution blocks:

$$y = \text{BinaryConv}(x) + \text{Shortcut}(x) \tag{2.17}$$

These skip connections serve as gradient highways, providing a direct path for gradient flow during backpropagation and significantly stabilizing training.

RPReLU (ReActNet)

ReActNet [12] replaces standard ReLU with RPReLU, a learnable activation with per-channel pre-shift and post-shift parameters:

$$\text{RPReLU}(x) = \text{PReLU}(x + \gamma) + \beta \tag{2.18}$$

where $\gamma \in \mathbb{R}^C$ and $\beta \in \mathbb{R}^C$ are learnable channel-wise shift parameters. This enables the binary network to learn asymmetric activation distributions, compensating for the information loss caused by binarization.

Libra-PB (Balanced Binarization)

IR-Net [17] proposed Libra Parameter Binarization (Libra-PB), which centres the weight distribution before applying the sign function:

$$w_b = \text{sign}(w - \bar{w}), \quad \bar{w} = \frac{1}{|\mathcal{F}|} \sum_{i \in \mathcal{F}} w_i \tag{2.19}$$

where $\bar{w}$ is the mean over the filter dimensions. This ensures approximately 50% of the binary weights are $+1$ and 50% are -1, maximizing the information entropy of the binary representation.

AdaBin (Adaptive Binary Scaling)

AdaBin [18] introduces a learnable per-channel output scaling factor α_c that modulates the binary convolution output:

$$y_c = \alpha_c \cdot \text{BinaryConv}_c(x) \tag{2.20}$$

At inference, α_c is fused into the subsequent BatchNorm layer's affine parameters, incurring zero additional computational cost.

2.5 Loss Functions for Segmentation

Semantic segmentation typically employs a combination of loss functions to handle class imbalance and optimize for region-based metrics.

2.5.1 Cross-Entropy Loss

The pixel-wise cross-entropy loss for C classes is:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{|\Omega|} \sum_{i \in \Omega} \sum_{c=1}^{C} y_{i,c} \log(\hat{y}_{i,c}) \tag{2.21}$$

where Ω is the set of valid pixels, $y_{i,c}$ is the ground-truth one-hot label, and $\hat{y}_{i,c}$ is the predicted probability after softmax.

2.5.2 Dice Loss

The Dice loss directly optimizes the Dice coefficient (F1 score), which is robust to class imbalance:

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{1}{C} \sum_{c=1}^{C} \frac{2 \sum_i y_{i,c} \hat{y}_{i,c} + \epsilon}{\sum_i y_{i,c} + \sum_i \hat{y}_{i,c} + \epsilon} \tag{2.22}$$

where ϵ is a smoothing constant to prevent division by zero.

2.5.3 Combined Cross-Entropy Dice Loss

The default loss function used in both UKAN and BiKA training combines both objectives:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \lambda_{\text{Dice}} \cdot \mathcal{L}_{\text{Dice}} \tag{2.23}$$

This jointly optimizes for per-pixel classification accuracy and region-level overlap, with the Dice component ensuring that small classes (e.g., lane markings) are not overwhelmed by the dominant background class.

2.6 Optimization and Activation Functions

The successful training of both full-precision and binarized networks relies heavily on the choice of optimization algorithms and activation functions. In our implementations across the UKAN and BiKA architectures, we employ several standard and specialized techniques.

2.6.1 Optimization Methods

For model optimization, we utilize variations of stochastic gradient descent. The Adam optimizer [25] computes adaptive learning rates for each parameter using estimates of first and second moments of the gradients, making it highly effective for complex architectures like KANs. To improve generalization, we also employ AdamW [26], which decouples weight decay from the gradient update, providing better regularization. During training, learning rates are dynamically adjusted using the OneCycleLR scheduler [27], a super-convergence technique that anneals the learning rate from an initial value to a high maximum value and then down to a minimum, preventing the model from settling into sharp, suboptimal local minima.

2.6.2 Activation Functions

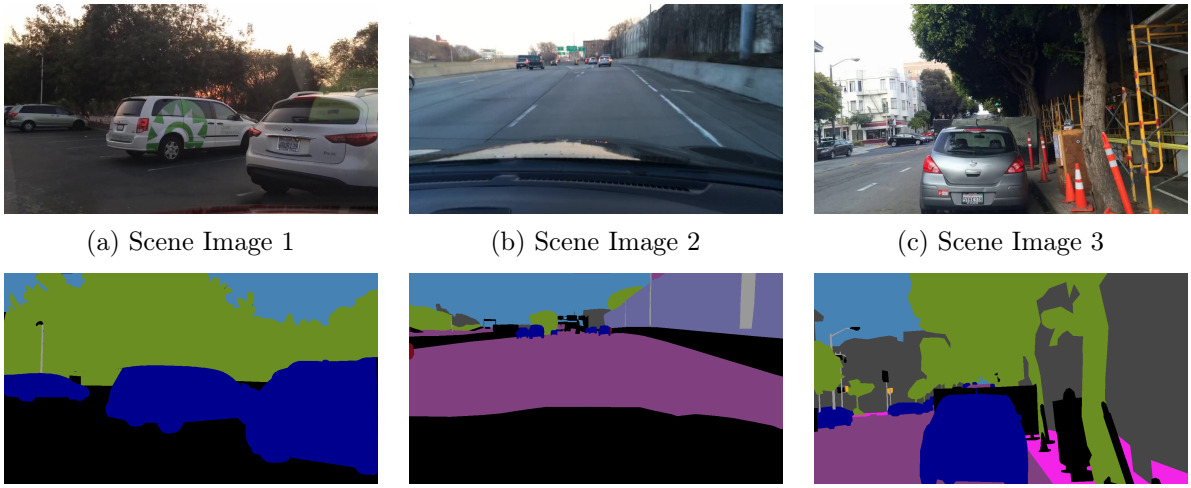
Activation functions inject the necessary non-linearity into the network. Beyond the standard Rectified Linear Unit (ReLU), our KAN variants utilize more advanced smooth activations. The Sigmoid Linear Unit (SiLU or Swish) [28], defined as $x \cdot \sigma(x)$, is used extensively as a base activation function across the KAN linear layers due to its smooth, non-monotonic properties that improve gradient flow. Additionally, Hardswish [21] is employed in specific KAN variants (HardSwish-KAN) to approximate the smooth SiLU activation with piecewise linear functions, significantly reducing computational overhead on edge devices while maintaining accuracy.

2.7 BDD100K Dataset

The Berkeley DeepDrive 100K (BDD100K) dataset [29] is one of the largest and most diverse driving scene benchmarks designed for multitask learning in autonomous driving perception. It contains 100,000 video clips captured across varied geographic locations, times of day (daytime, night, dawn/dusk), and weather conditions (clear, rainy, foggy, snowy).

For semantic segmentation, BDD100K provides pixel-level ground truth annotations for 10,000 high-resolution images (1280×720), partitioned into 7,000 training, 1,000 validation, and 2,000 test images. The annotations cover 19 evaluation semantic classes (plus background), including road, sidewalk, vehicle, pedestrian, rider, traffic sign, and sky. In this work, we utilize BDD100K to benchmark our segmentation networks under both multi-class and binary road segmentation setups.

Figure 2.3 displays a 2×3 grid showcasing sample input driving images alongside their corresponding color-coded ground truth semantic segmentation masks.



(a) Scene Image 1 (b) Scene Image 2 (c) Scene Image 3

(d) Ground Truth 1 (e) Ground Truth 2 (f) Ground Truth 3

Figure 2.3: Sample 2×3 grid from the BDD100K dataset: top row shows RGB driving scene images, bottom row shows the corresponding pixel-level ground truth semantic segmentation color labels [29].

Chapter 3

Methodology and Architecture Design

This chapter details the two KAN-based segmentation architectures: the full-precision UKAN and the binarized BiKASegNet. We describe the integration of KAN layers into the U-Net framework, the design of binarized KAN convolutions, and the knowledge distillation pipeline connecting both models.

3.1 UKAN: U-Shaped KAN Segmentation Network

UKAN adapts the U-Net architecture by replacing the standard MLP/Transformer blocks in the bottleneck and lower-resolution stages with KAN-based blocks, while retaining efficient CNN stages at higher resolutions where spatial dimensions are large.

3.1.1 Overall Architecture

The UKAN architecture follows a three-stage CNN encoder, a KAN encoder-bottleneck, and a mixed KAN-CNN decoder, as illustrated in Figure 3.1.

The encoder consists of three CNN stages (ConvLayer), each performing a double 3×3 convolution with BatchNorm and ReLU, followed by 2×2 max pooling. The channel dimensions progress as $3 \to C_0/8 \to C_0/4 \to C_0$, where C_0 is the base embedding dimension (default: 64).

3.1.2 KAN Encoder and Bottleneck

At the transition from CNN to KAN stages, PatchEmbed layers convert 2D feature maps into 1D token sequences via a strided convolution projection:

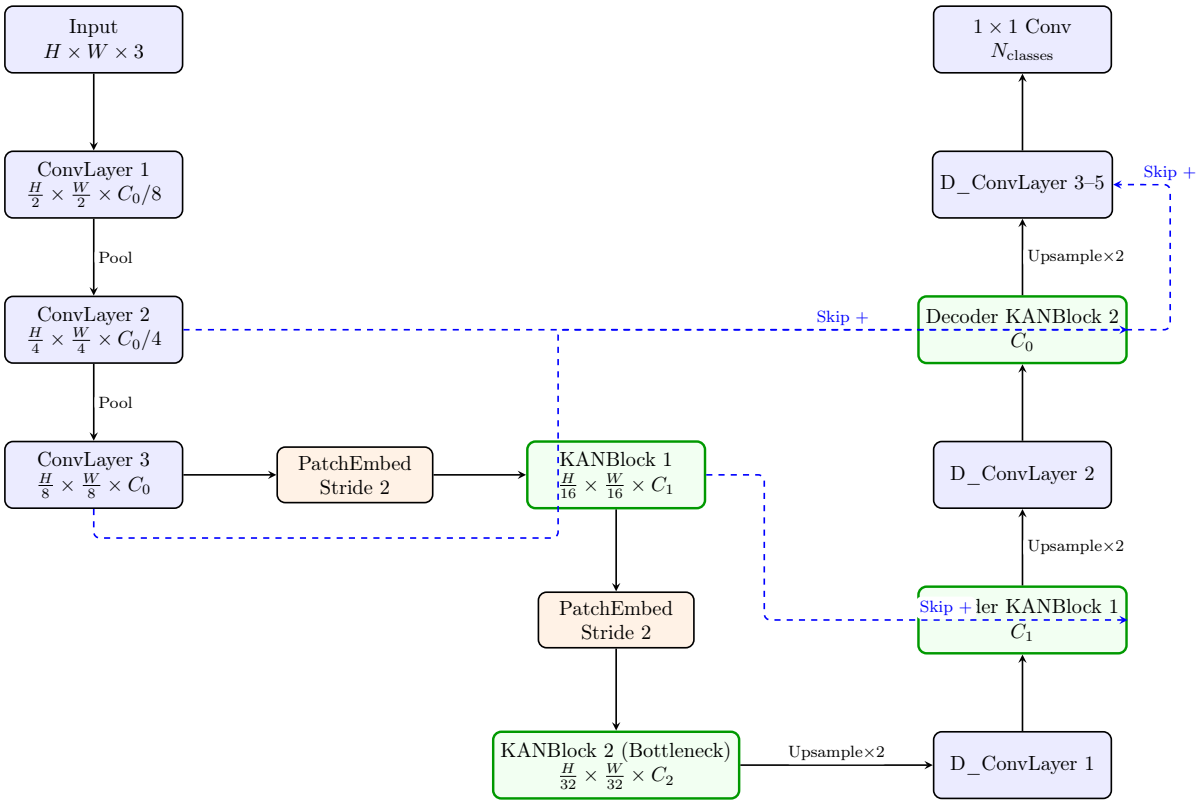
$$\mathbf{T} = \text{LayerNorm}\left(\text{reshape}\left(\text{Conv}_{k\times k,s}(\mathbf{F})\right)\right) \in \mathbb{R}^{B\times N\times C} \tag{3.1}$$

where $\mathbf{F}$ is the input feature map, k is the patch size, s is the stride, and $N = H' \times W'$ is the number of tokens. Two KAN encoder stages process tokens at embedding dimensions $C_1 = 128$ and $C_2 = 256$ (default configuration).

3.1.3 KANBlock Design

Each KANBlock follows a Transformer-style pre-norm residual pattern:

$$\mathbf{T}' = \mathbf{T} + \text{DropPath}\left(\text{KANLayer}(\text{LayerNorm}(\mathbf{T}), H, W)\right) \tag{3.2}$$



**Note: Green blocks indicate KAN-based stages. Blue blocks indicate standard CNN stages. Dashed lines indicate additive skip connections.*

Figure 3.1: UKAN Architecture: CNN encoder stages transition to KAN blocks at lower resolutions, with a mixed KAN-CNN decoder.

The KANLayer replaces the standard MLP (or self-attention) module. It consists of three KAN linear projections interleaved with depthwise convolution blocks (DW_bn_relu) for spatial token mixing:

$$\mathbf{T}_1 = \text{DW\_bn\_relu}\left(\text{KAN\_fc}_1(\mathbf{T}), H, W\right) \tag{3.3}$$
$$\mathbf{T}_2 = \text{DW\_bn\_relu}\left(\text{KAN\_fc}_2(\mathbf{T}_1), H, W\right) \tag{3.4}$$
$$\mathbf{T}_3 = \text{DW\_bn\_relu}\left(\text{KAN\_fc}_3(\mathbf{T}_2), H, W\right) \tag{3.5}$$

Each KAN_fc is an instance of the selected KAN linear variant (FasterKAN, ReLU-KAN, etc.) with grid size $G = 8$. The depthwise convolutions preserve spatial context since KAN operations act independently on the channel dimension. Each DW_bn_relu block performs:

$$\text{DW\_bn\_relu}(\mathbf{T}, H, W) = \text{ReLU}\left(\text{BN}\left(\text{DWConv}_{3\times3}(\text{reshape}(\mathbf{T}, H, W))\right)\right) \tag{3.6}$$

3.1.4 Decoder

The decoder combines KAN blocks with standard convolutional upsampling (D_ConvLayer). Each decoder stage: (1) applies a D_ConvLayer convolution to reduce channels, (2) bilinearly upsamples by factor 2, (3) adds the skip connection from the corresponding encoder stage, (4) processes with a decoder KANBlock. The final three stages use only D_ConvLayers to restore the original spatial resolution, followed by a 1×1 convolution head producing per-class logits.

3.2 BiKASegNet: Binarized KAN Segmentation Network

BiKASegNet implements the KAN philosophy of learnable per-connection activations in a radically different way: instead of smooth spline functions, it uses binarized step functions with learnable per-connection thresholds. This produces a network where both weights and activations are binary ($\{+1, -1\}$), enabling extreme computational efficiency.

3.2.1 BiKA_Conv2d: The Binarized KAN Convolution

The core innovation of BiKA is the BiKA_Conv2d layer. In a standard convolution, the weight tensor has shape $(C_{\text{out}}, C_{\text{in}}, K_H, K_W)$ and the bias (if present) has shape $(C_{\text{out}},)$—a single scalar per output channel. In BiKA_Conv2d, the bias has shape:

$$\mathbf{b} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times K_H \times K_W} \tag{3.7}$$

This is a **per-connection bias**—every single synapse in the convolution has its own learnable threshold. The forward computation is:

$$y_{o,h',w'} = \sum_{c,k_h,k_w} \text{sign}\left(w_{o,c,k_h,k_w} - \bar{w}_o\right) \cdot \text{sign}\left(x_{c,h'+k_h,w'+k_w} + b_{o,c,k_h,k_w}\right) \tag{3.8}$$

where $\bar{w}_o = \frac{1}{C_{\text{in}} K_H K_W} \sum_{c,k_h,k_w} w_{o,c,k_h,k_w}$ is the Libra-PB weight centering.

This design directly realizes the KAN principle: each edge has its own activation function $\phi_{o,c,k_h,k_w}(x) = \text{sign}(x + b_{o,c,k_h,k_w})$, where the threshold b is learnable. The weight binarization $\text{sign}(w - \bar{w})$ applies Libra-PB centering to ensure balanced binarization.

AdaBin Output Scaling

Following the binary convolution, an AdaBin learnable per-channel scaling factor is applied:

$$y_c \leftarrow \alpha_c \cdot y_c, \quad \alpha_c \in \mathbb{R}^{C_{\text{out}} \times 1 \times 1} \tag{3.9}$$

At inference, α_c is fused into BatchNorm's γ parameter, adding zero computational overhead.

Custom CUDA Implementation

The BiKA_Conv2d forward pass is implemented via a custom CUDA kernel that:

1. Packs binarized weights into `int32` words (32 binary weights per integer).
2. Pre-negates and converts the bias tensor to FP16 for shared memory bandwidth savings.
3. Computes the binary convolution using XNOR and popcount operations on the packed representations.

The backward pass uses the Straight-Through Estimator (STE) to propagate gradients through the sign functions, with gradients zeroed for weights outside the $[-1, 1]$ clipping window.

3.2.2 BiKAConvBlock: Enhanced Binary Block

Each BiKAConvBlock consists of two BiKA_Conv2d layers with BatchNorm and RPReLU activations, plus an FP32 skip connection:

$$\mathbf{s} = \text{Shortcut}(\mathbf{x}) \tag{3.10}$$

$$\mathbf{h} = \text{RPReLU}\left(\text{BN}\left(\text{BiKA\_Conv2d}_1(\mathbf{x})\right)\right) \tag{3.11}$$

$$\mathbf{y} = \text{RPReLU}\left(\text{BN}\left(\text{BiKA\_Conv2d}_2(\mathbf{h})\right) + \mathbf{s}\right) \tag{3.12}$$

The shortcut is a 1×1 FP32 convolution with BatchNorm when the channel dimensions change, and an identity otherwise. Activation checkpointing is applied during training to reduce VRAM usage by approximately 60%.

3.2.3 BiKASegNet Architecture

The full BiKASegNet follows a standard U-Net topology with BiKAConvBlocks replacing standard convolution blocks, as shown in Figure 3.2.

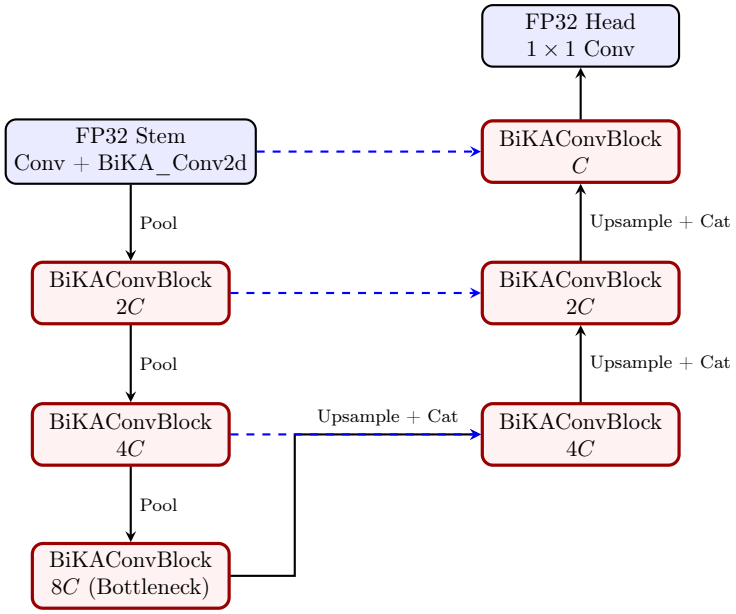


Figure 3.2: BiKASegNet Architecture: red blocks use binarized BiKA_Conv2d layers. The stem and classification head remain in full precision for gradient stability.

Key architectural decisions:

- **Full-precision stem**: The first encoder layer uses a standard FP32 convolution for the initial $3 \to C$ projection, followed by a BiKA_Conv2d. Binarizing the very first layer severely degrades accuracy since it directly quantizes the raw 8-bit RGB pixel values.
- **Full-precision head**: The final 1×1 classification convolution remains in FP32 to preserve the fine-grained logit distributions needed for accurate per-pixel classification.
- **Concatenation skip connections**: Unlike UKAN's additive skips, BiKASegNet concatenates encoder and decoder features before each decoder block, providing richer feature recombination.
- **Base channels**: The default base channel count is $C = 16$, keeping the binary model extremely lightweight.

3.3 Training Methodology

3.3.1 Optimisation for BNN Stability

Training BiKASegNet requires careful handling of the binary weight dynamics. Specifically:

- **Weight decay exclusion**: Weight decay is explicitly disabled for all BiKA_Conv2d parameters (weights and per-connection biases), BatchNorm affine parameters, and RPReLU shift parameters. Applying weight decay to binary thresholds causes them to collapse toward zero, annihilating the STE gradient and killing the backward pass entirely.
- **STE weight clamping**: Binary weights are optionally clamped to $[-1, 1]$ after each optimiser step to ensure they remain within the STE gradient window.
- **Bias initialisation**: Per-connection biases are uniformly initialized in $[-2.0, 0.5]$ to ensure diverse initial activation thresholds across connections.

3.3.2 Data Augmentation

Both UKAN and BiKA training pipelines employ the following augmentations via the Albumentations library:

- Random horizontal flip (probability 0.5).
- Random 90-degree rotation.
- CutMix augmentation for regularisation.
- Resize to the target input resolution (default: 256×256 or 512×512).
- ImageNet-standard normalisation ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$).

3.3.3 Training Infrastructure

Models are trained using PyTorch with Automatic Mixed Precision (AMP) for memory efficiency (FP16/BF16 forward pass with FP32 master weights). Multi-GPU training is supported via Distributed Data Parallel (DDP) with `torchrun`. Learning rate scheduling uses CosineAnnealingLR or OneCycleLR, and gradient accumulation is available for simulating larger effective batch sizes.

Chapter 4

Experimental Evaluation and Results

This chapter details the experimental setup, dataset preparation, and performance evaluation of UKAN and BiKASegNet on the BDD100K road segmentation benchmark.

4.1 Experimental Setup

All experiments were conducted on a Linux workstation equipped with NVIDIA GPUs. The software stack consists of Python 3.10, PyTorch 2.x, and the Albumentations augmentation library. Models were trained on the BDD100K dataset [29], which provides densely annotated driving scene images with per-pixel semantic labels covering road surfaces, lane markings, vehicles, pedestrians, and background categories. We evaluate in both a 2-class configuration (road foreground vs. background) and the full multi-class configuration.

4.2 Segmentation Quality Benchmarks

The primary evaluation metrics are mean Intersection over Union (mIoU) and mean Dice coefficient, computed via a confusion-matrix-based epoch-level accumulator to avoid batch-averaging inaccuracies.

The IoU for class c is computed as:

$$\mathrm{IoU}_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c + \mathrm{FN}_c} \tag{4.1}$$

and the Dice coefficient as:

$$\mathrm{Dice}_c = \frac{2 \cdot \mathrm{TP}_c}{2 \cdot \mathrm{TP}_c + \mathrm{FP}_c + \mathrm{FN}_c} \tag{4.2}$$

where TP_c, FP_c, and FN_c are the true positives, false positives, and false negatives for class c accumulated over the entire validation set.

Table 4.1 summarizes the segmentation performance of the evaluated architectures.

4.3 Model Efficiency Analysis

Table 4.2 compares the model size and computational efficiency of the evaluated architectures.

The theoretical compression ratio of BiKASegNet relative to a full-precision network of identical topology is approximately 32× for the binarized layers, since each 32-bit float weight is

Model	Precision	KAN Variant	mIoU (%)	Dice (%)
U-Net (Baseline)	FP32	–	–	–
UKAN (FasterKAN)	FP32	RSWAf	–	–
UKAN (ReLU-KAN)	FP32	ReLU	–	–
UKAN (PWLO-KAN)	FP32	PWLO	–	–
BiKASegNet	1-bit	Binary Step	–	–
BiKASegNet + KD	1-bit	Binary Step	–	–

**Note: Results to be filled from experimental runs. KD = Knowledge Distillation from UKAN teacher.*

Table 4.1: Segmentation accuracy comparison across architectures on BDD100K road segmentation.

Model	Params (M)	Size (MB)	FPS	Compression
U-Net (FP32)	–	–	–	1×
UKAN (FP32)	–	–	–	1×
BiKASegNet (1-bit)	–	–	–	$\sim 32\times$

**Note: Compression ratio is relative to the FP32 model of equivalent architecture. FPS measured on the evaluation GPU.*

Table 4.2: Model efficiency comparison: parameter count, storage size, inference throughput, and compression ratio.

replaced by a single bit. In practice, the full-precision stem and head slightly reduce the overall compression ratio.

4.3.1 Inference Throughput

The binary convolution operations in BiKA_Conv2d leverage XNOR and popcount operations through the custom CUDA kernel. Table 4.3 profiles the per-layer latency contributions.

Component	UKAN Latency (ms)	BiKA Latency (ms)
Encoder Stages	–	–
KAN/Binary Blocks	–	–
Decoder Stages	–	–
Classification Head	–	–
Total	–	–

**Note: Latency measured as mean over 1000 forward passes at* 512×512 *input resolution.*

Table 4.3: Per-component inference latency comparison between UKAN and BiKASegNet.

4.4 Ablation Studies

4.4.1 Effect of KAN Variant Selection

To isolate the impact of the KAN activation function choice, we trained UKAN with each variant using identical hyperparameters. The KAN variant selection affects both accuracy and inference

speed, as each variant trades off the smoothness of the learned activation against computational cost.

4.4.2 Effect of Knowledge Distillation

Table 4.4 examines the impact of knowledge distillation on BiKASegNet accuracy.

Configuration	**Teacher**	**mIoU (%)**	**Δ mIoU**
BiKA (no KD)	–	–	–
BiKA + KD ($\tau = 4$)	UKAN-FasterKAN	–	–
BiKA + KD + Boundary	UKAN-FasterKAN	–	–

**Note: Δ mIoU is relative to the no-KD baseline.*

Table 4.4: Ablation study on knowledge distillation and boundary loss for BiKASegNet.

4.4.3 Effect of BNN Training Techniques

We evaluate the individual contribution of each BNN accuracy-improvement technique integrated into BiKAConvBlock:

- **Libra-PB**: Weight centering for balanced binarization.
- **RPReLU**: Learnable activation shifts vs. standard ReLU.
- **FP32 Skip**: Bi-Real Net gradient highway connections.
- **AdaBin**: Learnable output scaling.

Disabling the legacy mode activates all four techniques simultaneously. In legacy mode, the network reverts to plain ReLU activations and no skip connections, establishing the minimal BNN baseline.

Chapter 5

Conclusions and Future Works

5.1 Conclusions

In this thesis, we implemented, evaluated, and compared two complementary approaches for integrating Kolmogorov-Arnold Networks into segmentation architectures for road scene understanding.

The UKAN architecture demonstrates that replacing standard MLP projections with KAN layers in the U-Net bottleneck produces competitive segmentation quality while offering greater representational flexibility through learnable per-edge activation functions. The availability of multiple KAN variants (FasterKAN, ReLU-KAN, HardSwish-KAN, PWLO-KAN, TeLU-KAN) enables architecture-level tuning of the accuracy-latency trade-off depending on the deployment target.

The BiKASegNet architecture demonstrates that the KAN philosophy of per-connection learnable activations can be realized through binarized step functions with learnable thresholds, achieving extreme compression ($\sim$32$\times$) and computational acceleration through XNOR/popcount operations. The integration of state-of-the-art BNN techniques (Libra-PB, AdaBin, RPReLU, FP32 skip connections) is essential for maintaining segmentation quality under binary constraints. Knowledge distillation from a full-precision UKAN teacher further bridges the binarization gap.

5.2 Future Works

To extend this research, several areas should be investigated:

- **Edge Deployment**: Deploy BiKASegNet on actual embedded platforms (Raspberry Pi, Jetson Nano) and benchmark real-world inference latency and power consumption.
- **Multi-class Segmentation**: Extend the evaluation from 2-class road segmentation to the full BDD100K label set (20+ classes) and evaluate how KAN integration affects fine-grained category discrimination.
- **Temporal Consistency**: Integrate the segmentation networks into a video pipeline with temporal smoothing to evaluate frame-to-frame consistency of road boundary predictions.
- **Hybrid KAN-BNN Architectures**: Explore architectures that selectively apply binarization only to specific encoder/decoder stages while keeping KAN-critical bottleneck layers in higher precision.

Bibliography

[1] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljačić, T. Y. Hou, and M. Tegmark, "KAN: Kolmogorov-arnold networks," *arXiv preprint arXiv:2404.19756*, 2024.

[2] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *International Conference on Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pp. 234–241, 2015.

[3] Y. Liu, S. Ullah, and A. Kumar, "BiKA: Kolmogorov-Arnold-Network-inspired ultra lightweight neural network hardware accelerator," *arXiv preprint arXiv:2602.23455*, 2026.

[4] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3431–3440, 2015.

[5] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. M. Alvarez, and P. Luo, "SegFormer: Simple and efficient design for semantic segmentation with transformers," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, pp. 12077–12090, 2021.

[6] H. Cao, Y. Wang, J. Chen, D. Jiang, X. Zhang, Q. Tian, and M. Wang, "Swin-Unet: Unet-like pure transformer for medical image segmentation," *arXiv preprint arXiv:2105.05537*, 2022.

[7] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, P. Dollár, and R. Girshick, "Segment anything," *arXiv preprint arXiv:2304.02643*, 2023.

[8] A. N. Kolmogorov, "On the representation of continuous functions of several variables by superposition of continuous functions of one variable and addition," *Doklady Akademii Nauk SSSR*, vol. 114, pp. 953–956, 1957.

[9] V. I. Arnold, "On the representation of functions of several variables as a superposition of functions of a smaller number of variables," *Selected Works*, 2009.

[10] C. Li, X. Liu, W. Li, C. Wang, H. Liu, and Y. Yuan, "U-KAN makes strong backbone for medical image segmentation and generation," *arXiv preprint arXiv:2406.02918*, 2024.

[11] Z. Liu, B. Wu, W. Luo, X. Yang, W. Liu, and K.-T. Cheng, "Bi-Real Net: Enhancing the performance of 1-bit CNNs with improved representational capability and advanced training algorithm," in *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 722–737, 2018.

[12] Z. Liu, Z. Shen, M. Savvides, and K.-T. Cheng, "ReActNet: Towards precise binary neural network with generalized activation functions," in *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 143–159, 2020.

[13] L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, and H. Adam, "Encoder-decoder with atrous separable convolution for semantic image segmentation," in *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 801–818, 2018.

[14] A. Tsiamis, "FasterKAN: Accelerating kolmogorov-arnold networks with RSWAf basis functions." `https://github.com/AthanasiosDelis/faster-kan`, 2024.

[15] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio, "Binarized neural networks: Training deep neural networks with weights and activations constrained to +1 or -1," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 29, pp. 4107–4115, 2016.

[16] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi, "XNOR-Net: Imagenet classification using binary convolutional neural networks," in *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 525–542, 2016.

[17] H. Qin, R. Gong, X. Liu, M. Shen, Z. Wei, F. Yu, and J. Song, "Forward and backward information retention for accurate binary neural networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2250–2259, 2020.

[18] Z. Tu, X. Chen, P. Ren, and Y. Wang, "AdaBin: Improving binary neural networks with adaptive binary sets," in *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 262–278, 2022.

[19] B. Zhuang, C. Shen, M. Tan, L. Liu, and I. Reid, "Structured binary neural networks for accurate image classification and semantic segmentation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 413–422, 2019.

[20] Q. Qiu, G. Xu, *et al.*, "ReLU-KAN: New kolmogorov-arnold networks that only need matrix addition, dot multiplication, and ReLU," *arXiv preprint arXiv:2406.02075*, 2024.

[21] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, and H. Adam, "Searching for MobileNetV3," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 1314–1324, 2019.

[22] A. Fernandez and A. Mali, "TeLU activation function for fast and stable deep learning," *arXiv preprint arXiv:2412.20269*, 2024.

[23] N. Schoots, M. J. Villani, and N. uit de Bos, "Relating piecewise linear kolmogorov arnold networks to ReLU networks," in *Proceedings of the 28th International Conference on Artificial Intelligence and Statistics (AISTATS)*, vol. 258, pp. 199–207, 2025.

[24] Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," in *arXiv preprint arXiv:1308.3432*, 2013.

[25] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, 2015.

[26] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations (ICLR)*, 2019.

[27] L. N. Smith and N. Topin, "Super-convergence: Very fast training of neural networks using large learning rates," in *Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications*, vol. 11006, p. 1100612, SPIE, 2019.

[28] P. Ramachandran, B. Zoph, and Q. V. Le, "Searching for activation functions," in *International Conference on Learning Representations (ICLR) Workshop*, 2018.

[29] F. Yu, H. Chen, X. Wang, W. Xian, Y. Chen, F. Liu, V. Madhavan, and T. Darrell, "BDD100K: A diverse driving dataset for heterogeneous multitask learning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2636–2645, 2020.